

Mathematica研究会

研究発表会 2025-10-11

天野晃

大規模ファイルの取り扱い

🐼 基本：Import[]がサポートする形式を利用する

🐼 基本的にImport[]は遅くない

Mathematica

```
data (1.3G)

In[38]:= AbsoluteTiming[csvf = Flatten[Import[basedir <> subdir <> file, "CSV"]];]
[実際にかった時間] [平滑化] [インポート]

Out[38]= {4.24184, Null}

In[39]:= Head[csvf]
[頭部]

Out[39]= List

In[5]:= totalcount = Length[csvf]
[長さ]

Out[5]= 270051911

In[6]:= Needs["Developer`"]
[存在しない場合に読み込む]

In[7]:= PackedArrayQ[csvf]

Out[7]= True

Matlab

>> ti ; ata=rea matrix('/V lumes/LaCie 1
経過時間は 21.297998 秒です。
>> |

OS(shell)
```

キャッシュ無し：

```
year % time cat all_pubyear.list > /dev/null
0.01s user 0.21s system 3% cpu 5.961 total
```

キャッシュあり：

```
byear % time cat all_pubyear.list > /dev/null
0.01s user 0.08s system 95% cpu 0.095 total
```

🐼 ただし、JSONやXMLは遅くなることがある

```
data (2.3G)

In[22]:= AbsoluteTiming[jsf = Flatten[Import[basedir <> subdir <> file, "JSONLines"]];]
[実際にかった時間] [平滑化] [インポート]

Out[22]= {172.558, Null}

In[25]:= jsf[[1]][[1]]

Out[25]= TabularRow[Column count: 2]

In[32]:= AbsoluteTiming[jsf = Import[basedir <> subdir <> filecsv, "CSV"]];]
[実際にかった時間] [インポート]

Out[32]= {5.60128, Null}

In[33]:= jsf // Length
[長さ]

Out[33]= 270051911

In[35]:= jsf // Head
[頭部]

Out[35]= List

In[40]:= PackedArrayQ[jsf]

Out[40]= True
```

🐼 テキストベース：TextでImport[]可能ならば、Import後に整形しても遅くならない（すでにメモリーに載っているため）

🐼 それ以外：外部コマンドでImport[]可能な形式に変換してから読み込む

🐼 計算機特性：計算機をshutdownするとメモリー上のファイルキャッシュ情報が失われ、ファイルアクセスが遅くなる。このため計算機はできるだけ起動したままにしておく。

メモリー管理

- 🐭 ガーベッジコレクション：Mathematicaはガーベッジコレクションを行っており、計算中およびデータロード中にメモリー使用が一時的に増大する。多くの場合一時的であれば物理メモリーを超える容量を使用しても問題ない。（経験的に1.5倍くらいまでOK）
- 🐭 OSによるメモリー管理の違い：
 - 🐭 Linux：自動圧縮機能はなく、ユーザーがswap領域の圧縮/非圧縮を設定する必要がある（zram）
 - 🐭 MacOS：自動圧縮機能が動作する
- 🐭 計算機特性：NUMAの場合、ノードメモリーを超える使用を行うとノード間通信が発生し、パフォーマンスが低下する => 長時間計算はあえて複数回に分けることも考慮する、さらにマイグレーション抑制によりパフォーマンス低下を抑えられる場合もある
- 🐭 メモリーのモニター：Mathematicaのレポート機能でなくOS備え付けの「アクティビティモニター」「タスクマネージャ」「numa系コマンド」を利用する方がメモリーだけでなく計算機全体の様子が把握できる、特にParallel系関数を利用する場合は外部からのモニターの方が容易となる